POS Integration Guide

Pawn Locker Integration — Shop / POS Vendor

Document	POS-INTEGRATION
Version	1.0
Date	2026-09-23
Audience	Pawn-shop / POS vendor
Gateway	v1.1 (drawerCode) — offline-first

Confidential — for integration use between the pawn-shop system and the locker provider.

Who this is for: the pawn-shop computer (C# POS, offline or LAN) and anyone sending bags to the locker through this gateway.

You have **two ways** to work with the gateway. Pick one: - **A. Web UI (recommended for offline shops):** open Store Bags, scan, send. No code needed. - **B. API (for LAN-connected POS):** `POST /api/v1/pos/...` with a Bearer token.

Swagger: `/docs/pos` (UI) + `/docs/pos.json` (OpenAPI). The locker callbacks are documented separately in `LOCKER-INTEGRATION.md`.

---

1. Concepts in plain language

You say	Gateway calls it	Example
List	`batchNo`	`ST-20260920-001` — one group of bags sent together
Bag number	`bagNo`	`BAG-00125` (from the bag QR)
Voucher number	`voucherNo`	`PV-2026-000123`
Voucher date	`voucherDate`	`2026-09-20` (YYYY-MM-DD)
Drawer	`drawerCode`	`D001` (v1.1; locker-approved, e.g. D001–D301). Old `location` like `C1-R1` still works for backward compat

Rules: - The **locker owns the current drawer**. If staff move a bag (`D001 → D301`), the gateway updates automatically. Your printed slip may be old — the locker always opens the current drawer. - Sending the **same list name twice never duplicates** bags. The second send returns `200 duplicate:true`. - If sending is interrupted, **send again with the same list name + same `X-Request-Id`**.

2. Way A — Web UI (offline shops, no code)

1. Log in → **Store Bags** → **+ Store new bags** (`/admin/storage-batches/create`).
2. List name is pre-filled (e.g. `ST-20260920-001`). Keep it or press New name.
3. For each bag, fill **Voucher / Date / Bag / Drawer**:
4. Click a **Bag** box, then scan the POS QR with a USB scanner (acts like a keyboard) or press the camera **Scan** button.

   Accepted QR formats (from POS `QRCode.Encode(LocationID + "," + MortgageInvoiceCode + "," + MortgageItemCode)`):

   - `D001,PV-2026-000123,BAG-00125` → Drawer, Voucher, Bag (POS order)
   - `BAG-00125,PV-2026-000123,D001` → Bag, Voucher, Drawer
   - `BAG-00125` alone → just fills the bag

- JSON `{"bagNo":"...","voucherNo":"...","drawerCode":"..."}`

6. Each scan **appends a new row** automatically.
7. Drawer can be picked from the dropdown (live free drawers from the locker) or typed as `D001`.
8. Press **Step 3 — Send to locker**. Result per bag: **Stored / Fix drawer / Waiting**.
9. If some need a fix: correct the drawer on the detail page and press **Try failed ones again**.
10. **Where is each bag?** → **Bags** page shows the current drawer (authoritative from locker).
11. **For your shop computer: Bags** → **Download for shop computer** exports CSV (`bag_no,voucher_no,current_drawer_code,...`) for USB import into C# (`examples/csharp-offline-sync/PosCsvImportService.cs`).
12. **Customer pickup:** shop prints the OUT QR (bag number only). If the shop printer is broken, open the bag → **Print pickup QR** fallback in this gateway.

3. Way B — API (LAN-connected POS)

Base URL: `http://<gateway-host>/api` (e.g. `http://192.168.1.50/api`). Never hard-code locker IPs in POS — only the gateway address.

Auth (all calls):

```
Authorization: Bearer {POS_API_TOKEN} Content-Type: application/json Accept:
application/json
```

Get the token from the gateway admin (Settings). The locker token is different — never use it here.

3.1 Send a list — `POST /api/v1/pos/storage-assignments`

Send one group of bags. For retries add a stable `X-Request-Id: POS-20260920-000001` header (keep it identical on retry).

```
curl -X POST http://192.168.1.50/api/v1/pos/storage-assignments \ -H "Authorization: Bearer
$POS_API_TOKEN" \ -H "Content-Type: application/json" \ -H "X-Request-Id:
POS-20260920-000001" \ -d '{ "batchNo": "ST-20260920-001", "items": [
{"voucherNo":"PV-2026-000001","voucherDate":"2026-09-20","bagNo":"BAG-001","drawerCode":"D001"},
{"voucherNo":"PV-2026-000002","voucherDate":"2026-09-20","bagNo":"BAG-002","drawerCode":"D002"}
] }'
```

Field rules: `batchNo` unique, `^[A-Za-z0-9-_]+$`, max 100; `items` 1–1000; `voucherDate` YYYY-MM-DD; `drawerCode` (preferred) or legacy `location`, `^[A-Za-z0-9-_]+$`, must be a locker-approved drawer (see Free Drawers page).

Responses: - `201` accepted → `{success:true, data:{batchNo, requestId, total, status, items:[...]}}` (gateway queues delivery to locker) - `200` duplicate → `{success:true, errorCode:"DUPLICATE_BATCH", data:{duplicate:true}}` — safe to ignore - `401` → wrong/missing POS token (`INTEGRATION_AUTH_FAILED`) - `422` → validation (`VALIDATION_ERROR` + `errors` object)

3.2 Look up a bag — `GET /api/v1/pos/bags/{bagNo}`

```
curl http://192.168.1.50/api/v1/pos/bags/BAG-00125 \ -H "Authorization: Bearer $POS_API_TOKEN" # 200 → {success:true, data:{bagNo, voucherNo, voucherDate, currentLocation, currentDrawerCode, lastLockerUpdateAt, lastLockerEventId}} # 404 → {success:false, errorCode:"BAG_NOT_FOUND"}
```

3.3 Current drawer only — `GET /api/v1/pos/bags/{bagNo}/location`

Same auth. Returns `{bagNo, currentLocation, currentDrawerCode, ...}`. Use this right before printing a pickup slip if you want the freshest drawer — but the locker opens the current drawer even with an old slip.

4. POS flows end-to-end

Store (end of day): POS collects bags → sends list (UI or §3.1) → gateway saves + forwards to locker (`POST {LOCKER_API_URL}/storage-assignments/batch` with same `batchNo` + `X-Request-Id`) → locker replies accepted/rejected per bag → gateway marks Stored / Fix drawer → operator puts bags in (kiosk: staff card → IN → scan bag QR → drawer opens).

Pickup (redemption): Customer pays at shop → shop prints OUT QR (**bag number only**, e.g. `BAG-00125`) → staff take slip to kiosk → staff card → OUT → scan QR → locker finds **current** drawer itself and opens → bag removed. No locker API call needed from POS.

Locker tidies up (consolidation): Locker staff move `D001 → D301` → locker calls gateway (see LOCKER-INTEGRATION.md) → gateway updates current drawer → shows under Bag Moves. Offline shops: re-download CSV if your local records must match.

5. Errors & retries (POS side)

What you see	Meaning	What to do
`DUPLICATE_BATCH` (200)	List already saved	Nothing — safe
`VALIDATION_ERROR` (422)	Missing/bad field	Fix highlighted field, resend
Item `REJECTED / INVALID_DRAWER_CODE`	Drawer not approved/occupied	Pick a free drawer (Free Drawers page), send only corrected bags as a **new** list
`LOCKER_UNAVAILABLE` / HTTP 0/500/503	Locker unreachable	Wait, then retry **same** `batchNo` + same `X-Request-Id` (3 tries, few seconds apart)
`401`	Token wrong	Check `POS_API_TOKEN`, not the locker token

Same `batchNo` + same `X-Request-Id` on every retry of the same logical list. Never invent a new list name just because the network timed out.

6. Checklist before go-live

- [ ] Can send a 10-bag list and see Stored / Partly stored correctly
- [ ] Invalid drawer (e.g. `D999`) shows Fix drawer with a clear message
- [ ] Same list sent twice → second is Duplicate, no double storage
- [ ] OUT QR (bag only) opens the correct current drawer after a move
- [ ] After locker consolidation `D001 → D301`, Bags page shows `D301`
- [ ] CSV download imports into your C# DB (if offline)